ALIGNMENT TECHNICAL REPORT ON QWEN2.5

HD
University of Illinois, Urbana-Champaign

ABSTRACT

The project is Alignment and Reasoning RL on **Qwen2.5-Math-1.5B** with different post-training methods. In this report, we will describe observations from different alignment experiments and conclude some practical skills that could be re-implemented in industrial pipelines. The source of this project is from **Standford University, CS336, assignment #5**.

1 BASELINE

Base model:Qwen2.5-Math-1.5B

	Accuracy	Format_match_rate
gsm8k	65.50%	79.00%
gsm8k[:50]	62%	74%
MATH	31.82%	55.08%

Table 1: Baseline for Qwen2.5

*Format_match means the output from LLM has `</think>...<answer>...</answer>` in it.

2 SUPERVISED FINE-TUNING

2.1 FULL FT

First, split the MATH dataset into train, validation, and test, with size of 7000, 500, and 5000 respectively. We then applied different training strategies on the train_set:
a. Use examples in the range {128, 256, 512, 1024}, along with using the full dataset.
b. filtered train data from Section 1.

In SFT, **learning_rate** = 5e-6, **batch_size**=4 (16 for gsm8k), **epoch**=3.

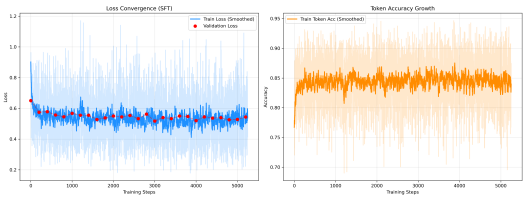

Figure 1: MATH, full dataset SFT

	Accuracy	Format_match_rate	Loss	step_per_epoch
MATH_size_128	32.34%	71.10%	0.5784	32
MATH_size_256	36.22%	79.58%	0.5786	64
MATH_size_512	41.36%	89.10%	0.5297	128
MATH_size_1024	42.38%	91.30%	0.5186	256
MATH	**42.60**%	91.62%	_	1750
MATH_filtered	41.66%	**91.88**%	0.5324	549
gsm8k w SFT_on_MATH	**71.57**%	98.03%	_	_
gsm8k	69.29%	99.17%	_	463

Table 2: Results

This experiment sampling to the original MATH training set using a Base Model, retaining only those samples that the model is able to answer correctly (Filtered Data), to verify that enhancing the purity of the training data can provide a robust logical pathways.

In Table 1, we can observe that Filtered SFT (41.66% accuracy) performed slightly worse than Full SFT (42.60%). Although the filtered data eliminated "noisy samples" that the model could not comprehend, the number of effective training steps plummeted from 1,750 to 549, resulting in a significant reduction in sample size.

This suggests that in tasks like MATH—which demand extremely high logical complexity—data diversity is often more critical than absolute correctness. The model is still able to learn useful mathematical expressions and logical structures from the challenging problems it initially failed to solve. The sample diversity provided by the full dataset compensated for the "noise" present in some of the data, thereby helping the model establish stronger generalization boundaries.

Another key finding of this experiment is the cross-task transfer capability facilitated by SFT. Although the model was fine-tuned exclusively on the MATH dataset, its performance on the GSM8K test set also saw a significant improvement:

- The precision increased from a baseline of 65.50% to 69.29%.
- The format success rate soared from 79.00% to 99.17%.

As a collection of elementary school math word problems, GSM8K can be viewed as a "simplified subset" of MATH (competition-level mathematics). Once the model mastered the more difficult and complex reasoning logic of MATH—along with its standardized CoT (Chain-of-Thought) format—its generalization capabilities naturally "spilled over" into simpler tasks. This result not only demonstrates the universality of mathematical reasoning skills but also illustrates how rigorous format alignment can effectively activate a model's intrinsic knowledge reserves.

2.2 LoRA

Due to the limit computing resources, LoRA is skiped in the project.

3 EXPERT ITERATION WITH VERIFIED REWARDS

3.1 METHODOLOGY

In this task, we implemented Expert Iteration. The algorithm is shown as the following:

Input: initial policy model $\pi_{\theta_{\text{init}}}$; reward function R; task questions $\mathcal{D}$

policy model $\pi_\theta \leftarrow \pi_{\theta_{\text{init}}}$
for *step* $= 1, \ldots, n_ei_steps$ **do**
  Sample a batch of questions $\mathcal{D}_b$ from $\mathcal{D}$
  Set the old policy model $\pi_{\theta_{\text{old}}} \leftarrow \pi_\theta$
  Sample G outputs $\{o^{(i)}\}_{i=1}^{G} \sim \pi_{\theta_{\text{old}}}(\cdot \mid q)$ for each question $q \in \mathcal{D}_b$
  Compute rewards $\{r^{(i)}\}_{i=1}^{G}$ for each sampled output $o^{(i)}$ by $R(q, o^{(i)})$
  Filter out wrong outputs (i.e., $o^{(i)}$ with $r^{(i)} = 0$) to obtain a dataset $\mathcal{D}_{\text{sft}}$ of correct question-response pairs
  $\pi_\theta \leftarrow \text{SFT}(\pi_\theta, \mathcal{D}_{\text{sft}})$
end
return π_θ

3.2 PERFORMANCE

In the data sampling part, the alrogithm will randomly choose data from the train dataset, then do the sampling progress. The default hyperparameters for in the forward and backward part is same as them in SFT in Section 2 with running 5 EI-iterations. Besides default setting, we also tried gradient acceleration for 10 EI-iterations.

	MATH		GMS8K	
	Accuracy	Format Rate	Accuracy	Format rate
SFT (Best)	0.426	91.62%	0.716	98.08%
SFT (Filtered)	0.417	91.88%	_	_
5-Step EI (step_5)	0.464	97.12%	0.732	99.77%
5-Step EI (Best)	0.452	93.68%	0.715	99.24%
10-Step EI (step_10)	0.454	97.16%	0.732	99.39%
10-Step EI (Best)	0.427	91.86%	0.717	98.03%

Table 3: Expert Iteration Result

4 GRPO

4.1 MATHEMATICAL FUNDAMENTALS

Group Relative Policy Optimization (GRPO) is a reinforcement learning (RL) algorithm used to train large language models (LLMs), notably in DeepSeek-R1 and DeepSeekMath, which was first appear in the technical report of DeepSeekMathShao et al. (2024). The key equation of GRPO is:

$$J_{\text{GRPO-Clip}}(\theta) = \mathbb{E}_{q\sim\mathcal{D},\{o^{(i)}\}_{i=1}^{G}\sim\pi_\theta(\cdot|q)}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o^{(i)}|}\sum_{t=1}^{|o^{(i)}|}\underbrace{\min\left(\frac{\pi_\theta(o_t^{(i)}|q,o_{<t}^{(i)})}{\pi_{\theta_{\text{old}}}(o_t^{(i)}|q,o_{<t}^{(i)})}A^{(i)},\text{clip}\left(\frac{\pi_\theta(o_t^{(i)}|q,o_{<t}^{(i)})}{\pi_{\theta_{\text{old}}}(o_t^{(i)}|q,o_{<t}^{(i)})},1-\epsilon,1+\epsilon\right)A^{(i)}\right)}_{\text{per-token objective}}\right] \tag{1}$$

In this project, we will apply **off-policy** GRPO, with **G**=[8,16] and sampling 32 questions each step with [200 for G=8, 40 for G=16] steps in total. The RL algorithm following:

Input: initial policy model $\pi_{\theta_{\text{init}}}$; reward function R; task questions $\mathcal{D}$

policy model $\pi_\theta \leftarrow \pi_{\theta_{\text{init}}}$;
for *step* $= 1, \ldots, n_grpo_steps$ **do**
  Sample a batch of questions $\mathcal{D}_b$ from $\mathcal{D}$;
  Set the old policy model $\pi_{\theta_{\text{old}}} \leftarrow \pi_\theta$;
  Sample G outputs $\{o^{(i)}\}_{i=1}^{G} \sim \pi_{\theta_{\text{old}}}(\cdot|q)$ for each question $q \in \mathcal{D}_b$;
  Compute rewards $\{r^{(i)}\}_{i=1}^{G}$ for each sampled output $o^{(i)}$ by running reward function $R(q, o^{(i)})$;
  Compute $A^{(i)}$ with group normalization (Eq. 28);
  for *train step* $= 1, \ldots, n_train_steps_per_rollout_batch$ **do**
    Update the policy model π_θ by maximizing the GRPO-Clip objective (Eq. 1);
  end
end
return π_θ

4.2 Performance

base model	MATH Accuracy	Format Rate	GMS8K Accuracy	Format rate
Base_Model, G=8	0.426	91.62%	0.716	98.08%
SFT, G=8	0.426	91.62%	0.716	98.08%
5-Step EI, G=8	0.432	94.08%	_	_
5-Step EI, G=16	0.443	94.78%	0.7074	95.22%

Table 4: GRPO Result

From Table 4, we can see that after apply GRPO on the model after SFT, the performance in improved. However, the accuracy and format_rate dropped if we apply GRPO on the model after Expert_iterations. Some possible reasons are:

- GRPO without KL constraints lead the model to sacrifice the desirable structure established by EI in pursuit of higher scores.
- When the model is already highly capable, a smaller Group Size results in noisy advantage calculations, which potentially causing the model to "disrupt" its correct logic.
- Data quality is better in EI, if the reward model is not that strong in GRPO, the model might reinforce wrong reasoning.

5 DPO

5.1 ALGORITHM

Input: initial SFT policy π_{sft}; preference dataset $\mathcal{D} = \{q, y_w, y_l\}$

Policy model $\pi_\theta \leftarrow \pi_{\text{sft}}$;
Reference model $\pi_{\text{ref}} \leftarrow \pi_{\text{sft}}$ (frozen);
Set KL coefficient β and learning rate η;

for *step* $= 1, \ldots,$ *n_dpo_steps* **do**
  Sample a batch of preference pairs (q, y_w, y_l) from $\mathcal{D}$;
  $L_w \leftarrow \log \frac{\pi_\theta(y_w|q)}{\pi_{\text{ref}}(y_w|q)} \quad L_l \leftarrow \log \frac{\pi_\theta(y_l|q)}{\pi_{\text{ref}}(y_l|q)}$;
  $\mathcal{L}_{\text{DPO}} = -\mathbb{E}\left[\log \sigma\left(\beta L_w - \beta L_l\right)\right]$;
  Update the policy model π_θ by minimizing $\mathcal{L}_{\text{DPO}}$ via gradient descent;
end

return π_θ

5.2 TRAINING

Due to the limited computing resources, the DPO training is skipped.

6 CONCLUSION

From this project, we can observe that the performance of Qwen2.5-MATH-1.5B increased from 31% on MATH and 65.5% on GSM8K to 50% and 77% respectively, which we considered as a significant improvement, demonstrating that leveraging high-quality data can effectively unlock the latent reasoning potential of a small-parameter base model.

The significant jump in accuracy and reasong format (e.g., +19% on MATH) marks a highly successful alignment phase, effectively distilling complex reasoning paths into the model. However, reaching this high baseline also established a plateau where subsequent RL-based optimization (GRPO) faced diminishing returns and high sensitivity to gradient noise.

In actual industrial practice, the capacity of the model should be better, how to make high-quality and adjust training strategy by analyzing loss and performance is crucial. Besides, parallel training is also a key skill that I should learn in the next internship.

REFERENCES

Tatsunori Hashimoto and Percy Liang. CS 336: Language Modeling from Scratch (Spring 2025). `https://cs336.stanford.edu/spring2025/`, 2025.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, YK Li, Y Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.